

MACHINE LEARNING IN FINANCE

MODULE 3 | LESSON 2

BAYESIAN STATISTICS IN PRACTICE

Reading Time	15 minutes
Prior Knowledge	Naïve Bayes classifier methodology, likelihood, prior.
Keywords	Gaussian Distributions, Model fitting, Likelihood, Prior distribution

In the previous lesson, we looked at how Bayesian statistics can be used to assign the probability of an outcome occurring. We looked at categorical features and will now look at continuous features to predict the future one-month return of a group of stocks exceeding the median.

1. Naïve Bayes in Practice

Recall from Module 3, Lesson 1, that posterior distribution can be expressed as a product of the likelihoods of the predictors given the class and the prior distributions of the class, as in (1.4). This is based on the assumption of conditional independence. This is true for predictors that are numeric and continuous as well since these predictors also have likelihood distributions given a specific target class.

$$\begin{aligned} P(Y=k \mid X_{\{1\}}, X_{\{2\}}, \cdots, X_{\{n\}}) &= \frac{P(X_{\{1\}}|Y=k)P(X_{\{2\}}|Y=k)\cdots P(X_{\{n\}}|Y=k)P(Y=k)}{\sum_j P(Y=j)\prod_{i=1}^n P(X_{\{i\}}|Y=j)} \tag{2.1} \end{aligned}$$

Let's consider a case where the likelihood distribution of the predictor follows a Gaussian distribution for each class. The predictor would be a strong explanatory variable, should the distributions differ. A strong predictor, which is often what we aim for, has very little overlap between the likelihoods for each class, thus differentiating between classes. This is shown in figure 1 for both a strong and weak predictor. The top plot shows the likelihood for a strong predictor given the target class as there is little overlap between the distributions (the two curves). The bottom plot shows a weaker predictor with significantly more overlap.

Fig 1: Strong vs. Weak Predictor Likelihoods

To put these ideas into perspective, let us look at an example where we apply the Naïve Bayes classifier to a stock returns problem.

In this section, we use data from Coqueret and Guida (chapter 9, section 9.4), specifically the future one-month returns data for a collection of stocks and financial data. This data can be accessed via <https://github.com/shokru/mlfactor.github.io/tree/master/material>. For illustrative purposes, we take a subset of stocks and features to predict the future one-month price being above the median, thus a classification problem. The features we look into are:

- *Mkt_Cap_12M_Usd* - Size
- *Pb* - Price to book
- *Vol1Y_Usd* - For share turnover
- *Return_On_Capital*
- *Roe* - Return on equity
- *Pe* - Price to Earnings ratio

We perform the analysis using the Python statistical software. Let us load the necessary libraries and import the data. We'll work with data from the year 2000 to 2018 and only 50 stock IDs.

```
In [ ]: import numpy as np
import pandas as pd

df_50 = pd.read_csv("mlfac_dat.csv")

# consider data from 2000 to 2018

df_50 = df_50[(df_50["date"] > "1999-12-31") & (df_50["date"] < "2019-01-01")]

# Filter on the first 50 stock ids

df_50 = df_50[df_50["stock_id"] <= 50]
```

The objective is to predict whether the future one-month returns are above the median. Hence, we can simply flag instances when it is above the median as 1 and 0 otherwise.

```
In [ ]: # Created a class variable for 1-Month returns if > median

df_50["R1M_Usd_C"] = np.where(df_50["R1M_Usd"] > df_50["R1M_Usd"].median(), 1, 0)

# Make the response variable into integer format

df_50["R1M_Usd_C"] = df_50["R1M_Usd_C"].astype(int)
```

Specify the predictors or features used in the classifier. These features have been transformed into variables that follow a uniform distribution. In practice, we can hope for Gaussian-distributed variables. Therefore, we will transform each feature using the Box-Muller transformation (Box and Muller 610).

```

In [ ]: # create a copy of data to store Gaussian converted data

df50_norm = df_50.copy()

# Features of interest
feature_cols = ["Mkt_Cap_12M_Usd", "Pb", "Vol1Y_Usd", "Return_On_Capital", "Roe"]

# Loop through Box-Muller for each data point for each predictor
for col in feature_cols:
    for i in np.arange(len(df_50)):
        # 1. Use U1 as current data value. Generate U2, which is Unif(0, 1)

        u1s, u2s = (
            df_50.loc[i, col],
            np.random.uniform(low=0.0, high=1.0, size=1)[0],
        ) # X.loc[i, col]

        # 2. Transform U1 to s

        ss = -np.log(u1s)

        # 3. Transform U2 to theta

        thetas = 2 * np.pi * u2s

        # 4. Convert s to r

        rs = np.sqrt(2 * ss)

        # 5. Calculate x and y from r and theta

        xs, ys = rs * np.cos(thetas), rs * np.sin(thetas)

        # 6. Store only one of the Gaussian derived values

        df50_norm.loc[i, col] = ys

```

It is worth looking at the likelihoods given the target class to gauge how strong or weak we can expect the predictors to be. We do this using the code below:

```

In [ ]: import matplotlib.pyplot as plt
import seaborn as sns

fig, axes = plt.subplots(2, 3, figsize=(18, 10), sharey=True)

i = 0

sns.kdeplot(ax=axes[0, i], data=df50_norm, x=feature_cols[i], hue="R1M_Usd_C")

i = 1

sns.kdeplot(ax=axes[0, i], data=df50_norm, x=feature_cols[i], hue="R1M_Usd_C")

i = 2

sns.kdeplot(ax=axes[0, i], data=df50_norm, x=feature_cols[i], hue="R1M_Usd_C")

i = 0

```

```

sns.kdeplot(ax=axes[1, i], data=df50_norm, x=feature_cols[i + 3], hue="R1M_Usd_C
i = 1

sns.kdeplot(ax=axes[1, i], data=df50_norm, x=feature_cols[i + 3], hue="R1M_Usd_C
i = 2

sns.kdeplot(ax=axes[1, i], data=df50_norm, x=feature_cols[i + 3], hue="R1M_Usd_C

```

Fig. 2: Likelihood for All 6 Features Given the Respective Target Class

Looking at the output in figure 2 and comparing it to figure 1, we can see the predictive power of the features are quite weak considering the significant overlap in the likelihoods. We can therefore expect poor model performance of the classifier.

We then load the library needed to split the dataset into a training and testing dataset to train and evaluate the classifier, respectively. An 80/20 train/test split is used. We consider only the features mentioned above.

```

In [ ]: # split dataset in features and target variable

X = df50_norm[feature_cols] # Transformed Features

y = df50_norm["R1M_Usd_C"] # Target variable

# Split dataset into training set and test set 80/20

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

```

Let us fit the Naïve Bayes classifier on the training set and obtain predictions on the unseen test set.

```

In [ ]: # initialize and fit classifier to training set
from sklearn.naive_bayes import GaussianNB

classifier = GaussianNB()

classifier.fit(X_train, y_train)

# get the predictions

y_pred = classifier.predict(X_test)

# Extract the probabilities of Target Class = 1.

y_predprobs = classifier.predict_proba(X_test)

probs = y_predprobs[:, 1]

# print the accuracy

```

```

from sklearn import metrics

# Model Accuracy, how often is the classifier correct?

print("Accuracy:", metrics.accuracy_score(y_test, y_pred))

```

The ROC curve and AUC can be obtained using the code below. It is useful to also show a no-skill or random guess classifier in the plot to compare to the Naïve Bayes classifier.

```

In [ ]: # plot the roc curve for the model
from sklearn.metrics import roc_auc_score, roc_curve

# generate a no skill or random guess prediction

ns_probs = [0 for _ in range(len(y_test))]

# calculate roc curves

ns_fpr, ns_tpr, _ = roc_curve(y_test, ns_probs)

# calculate scores

ns_auc = roc_auc_score(y_test, ns_probs) # random guess

tree_auc = roc_auc_score(y_test, probs) # tree classifier

# summarize scores

print("No Skill: ROC AUC=%.3f" % (ns_auc))

print("Classifier: ROC AUC=%.3f" % (tree_auc))

plt.plot(ns_fpr, ns_tpr, linestyle="--", linewidth=4, label="No Skill")

plt.plot(
    ns_fpr,
    ns_tpr,
    linewidth=2,
    label="NB Classifier",
)

# axis labels

plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

# show the legend

plt.legend()

# show the plot

plt.show()

```

Fig. 3: ROC Curve of Naïve Bayes Classifier Performance along with a Random Guess or No-Skill Model

The corresponding ROC curve is shown in figure 3. The AUC for this model is ~50%, thus confirming our suspicion of its weak predictive power.

2. Conclusion

The example covered in this lesson helps us understand the Naïve Bayes classifier by seeing it in practice. The model performance, however, is not as good as we would hope. This lesson focused on showing how to apply Naïve Bayes to a real-world classification problem using continuous predictors and how Naïve Bayes can easily be applied to categorical variables. To obtain a better model, stronger predictors are needed, i.e., explanatory variables that have different likelihood distributions for the target classes. Explanatory data analysis, feature selection, and engineering cannot be understated when dealing with modeling.

In the next section, we will look at tree-based algorithms, another modeling technique. The Naïve Bayes model assumes the features are conditionally independent of the target variable; however, tree-based algorithms do not require any assumptions about the underlying distribution. Both Naïve Bayes and trees do not require feature scaling, although tree-based algorithms are not sensitive to outliers. The prediction approach of trees are to place decision boundaries in the data whereas Naïve Bayes aims to model how the data were generated. These are known as discriminative and generative models, respectively. We will look at trees in depth in the next lesson.

References

- Box, G. E. P., and Mervin E. Muller. "A Note on the Generation of Random Normal Deviates." *The Annals of Mathematical Statistics*, vol. 29, no. 2, 1958, pp. 610–11, <https://doi.org/10.1214/aoms/1177706645>.
- Coqueret, Guillaume, and Tony Guida. *Machine Learning for Factor Investing: R Version*. Financial Mathematics Series. Chapman and Hall/CRC, 2020.